

Dissecting Instruction Level Parallelism

Instruction Level Parallelism (ILP) technology is often used to improve the performance of embedded processors in application-specific domains like Digital Signal Processing. Today's VLIW (Very Large Instruction Word) DSP architectures are specially designed to take advantage of ILP. This article aims to provide an insight into the technique behind Instruction Level Parallelism that will help compiler developers, in particular, to understand what ILP is, and how it can be designed while developing a compiler.

Instruction Level Parallelism (ILP) refers to the execution of two or more instructions in parallel – i.e., in the same instruction cycle. For example, consider the sample set of instructions for a hypothetical processor in Figure 1.

R0, R1, R2 and R3 represent general-purpose registers, and AR0 represents the address register. So *MOVE R0, [AR0]++* means that the contents of memory location AR0 have to be moved to register R0, and the address register AR0 has to be incremented.

Now, suppose that the hypothetical architecture under consideration supports parallelism of an arithmetic operation instruction and a transfer instruction (instructions '2' and '3' in Figure 1, also identified by in-line comments). We may execute these instructions in parallel, to achieve ILP, as shown in Figure 2.

Assuming that our hypothetical processor takes one instruction cycle and one word for any type of instruction, the reduction in the number of instruction cycles and code size with ILP is visible in Figure 3.

Internals of ILP: The algorithm

Note that we will restrict this article to achieve ILP within a single basic block only.

The ILP module is broken down into several sub-modules, as shown in Figure 4.

The overall algorithm to achieve ILP is:

1. Detect data dependencies
2. Construct transitive closure
3. Add architecture restrictions
4. Construct the ILP graph
5. Schedule and apply ILP

Each of these steps is explained in detail below. We use the following code block as the subject (to which we will apply the ILP algorithm, step by step):

```
1. MOVE R0, [AR0];
2. ADD R1, R2, R2;
3. MOVE R3, [AR1];
4. ADD R4, R3, R2;
5. ADD R3, R4, R4;
6. SUB R3, R3, R4
```

Step #1: Detect data dependency

An Instruction J is said to be (directly) data-dependent on an Instruction I, if any of the following relationships exists between the two instructions:

1. Data flow dependency
2. Anti-dependency
3. Output-dependency